CATEGORY

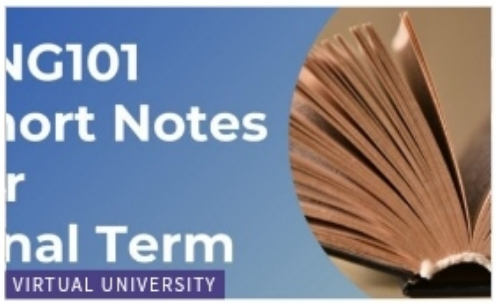
Virtual University

Pak301 Files for Final Term | 100% Prefect Files

Usmanmani - March 8, 2022

PHY101 Notes for Final Term Papers By Concepts Builder

UzmaAameer - March 6, 2022



Eng101 Final Term Notes By Concepts Builder

Ayesha Mustafa - March 5, 2022

ISL202 Complete Handouts For Vu Students

UzmaAameer - February 21, 2022

Latest news

Role of Technology in Education in 2022 | Technology Importance

Pak301 Files for Final Term | 100% Prefect Files

PHY101 Notes for Final Term Papers By Concepts Builder

How Internet Play Key Role in Students Life?

Eng101 Final Term Notes By Concepts Builder

Module 108
108. Algorithm: Software Verification Examples

Software Verification?

- ✓ Distinction between Program believed to be correct and the program that is correct.

Example #1 – Software Failure

- ✓ Fatal Therac-25 X-ray Radiation
- ✓ In 1986, a man in Texas received between 16,500-25,000 radiations in less than 10 sec, over a area of about 1 cm.
- ✓ He passed away 5 months later.
- ✓ The root cause of the incident was a SW failure

Example #2 – Welsch NHS IT Failure

- ✓ In 2018, doctors and staff were unable to access patients files in UK.
- ✓ This was a technical issue of software not the security issue.
- ✓ Hospitals and GP surgeries were affected.

Module 109
109. Programming Languages: Early Generations-I

As we learned in Figure 46, programs for modern computers consist of sequences of instructions that are encoded as numeric digits. Such an encoding system is known as a machine language. Unfortunately, writing programs in a machine language is a tedious task that often leads to errors that must be located and corrected (a process known as debugging) before the job is finished.

In the 1940s, researchers simplified the programming process by developing notational systems in which instructions could be represented in mnemonic rather than numeric form. For example, the instruction move the contents of register 5 to register 6 would be expressed as:

4056

using the machine language introduced in Figure 46, whereas in a mnemonic system it might appear as:

MOV R5, R6

As a more extensive example, the machine language routine

156C
166D
5056
306E
C000

which adds the contents of memory cells 6C and 6D and stores the result at location 6E (Figure 4 might be expressed as

LD R5,Price
LD R6, ShippingCharge
ADDI R0,R5 R6
ST R0,TotalCost
HLT

using mnemonics. (Here we have used LD, ADDI, ST, and HLT to represent *load*, *add*, *store*, and *h* Moreover, we have used the descriptive names Price, ShippingCharge, and TotalCost to refer to the memo

Copyright Virtual University of Pakistan

An assembler can convert the code into machine language is called assembler.

Module 110

110. Programming Languages: Early Generations-II

In the continuation of the previous module.

Once such a mnemonic system was established, programs called **assemblers** were developed to convert mnemonic expressions into machine language instructions. Thus, rather than being forced to develop a program directly in machine language, a human could develop a program in mnemonic form and then have it converted into machine language by means of an assembler.

A mnemonic system for representing programs is collectively called an **assembly language**. At the time assembly languages were first developed, they represented a giant step forward in the search for better programming techniques. In fact, assembly languages were so revolutionary that they became known as second-generation languages, the first generation being the machine languages themselves.

Although assembly languages have many advantages over their machine language counterparts, they still fall short of providing the ultimate programming environment. After all, the primitives used in an assembly language are essentially the same as those found in the corresponding machine language. The difference is simply in the syntax used to represent them. Thus, a program written in an assembly language is inherently machine dependent – that is, the instructions within the program are expressed in terms of a particular machine's attributes. In turn, a program written in assembly language cannot be easily transported to another computer design because it must be rewritten to conform to the new computer's register configuration and instruction set.

1) Code is Machine dependent

Example
TotalCost = Price + shipping charge

Another disadvantage of an assembly language is that a programmer, although not required to code instructions in numeric form, is still forced to think in terms of the small, incremental steps of the machine's language. The situation is analogous to designing a house in terms of boards, nails, bricks, and so on. It is true that the actual construction of the house ultimately requires a description based on these elementary pieces, but the design process is easier if we think in terms of larger units such as rooms, windows, doors, and so on.

In short, the elementary primitives in which a product must ultimately be constructed are not necessarily the primitives that should be used during the product's design. The design process is better suited to the use of high-level primitives, each representing a concept associated with a major feature of the product. Once the design is complete, these primitives can be translated to lower-level concepts relating to the details of implementation.

Following this philosophy, computer scientists began developing programming languages that were more conducive to software development than were the low-level assembly languages. The result was the emergence of a third generation of programming languages that differed from previous generations in that their primitives were both higher level (in that they expressed instructions in larger increments) and **machine independent** (in that they did not rely on the characteristics of a particular machine). The best-known early examples are FORTRAN (FORmula TRANslator), which was developed for scientific and engi- neering applications, and COBOL (COmmon Business-Oriented Language), which was developed by the U.S. Navy for business applications.

In general, the approach to third-generation programming languages was to identify a collection of high-level primitives in which software could be developed. Each of these primitives was designed so that it could be implemented as a sequence of the low-level primitives available in machine languages. For example, the statement

assign TotalCost the value Price + ShippingCharge

REDMI NOTE 8
AI QUAD CAMERA

The difference b/w compiler and interpreter is that when a Program is written Compiler first translate all of the code into machine language and then it executes however interpreter first translate the one instruction into machine language and execute that instruction then move to next one.

expresses a high-level activity without reference to how a particular machine should perform the task, yet it can be implemented by the sequence of machine instructions discussed earlier. Thus, our pseudocode structure

identifier = expression

is a potential high-level primitive.

Translater also known as compiler.

Once this collection of high-level primitives had been identified, a program, called a **translator,** was written that translated programs expressed in these high-level primitives into machine-language programs. Such a translator was similar to the second-generation assemblers, except that it often had to compile several machine instructions into short sequences to simulate the activity requested by a single high-level primitive. Thus, these translation programs were often called **compilers.**

An alternative to translators, called **interpreters,** emerged as another means of implementing third-generation languages. These programs were similar to translators except that they executed the instructions as they were translated instead of recording the translated version for future use. That is, rather than producing a machine-language copy of a program that would be executed later, an interpreter actually executed a program from its high-level form.

understand what user has written of a programming language try to in a language which is more familiar with human language

As a side issue, we should note that the task of promoting third-generation programming languages was not as easy as might be imagined. The thought of writing programs in a form similar to a natural language was so revolutionary that many in managerial positions fought the notion at first. Grace Hopper, who is recognized as the developer of the first compiler, often told the story of demonstrating a translator for a third-generation language in which German terms, rather than English, were used. The point was that the programming language was constructed around a small set of primitives that could be expressed in a variety of natural languages with only simple modifications to the translator. But she was surprised to find that many in the audience were shocked that, in the years surrounding World War II, she would be teaching a computer to "understand" German. Today we know that understanding a natural language involves much, much more than responding to a few rigorously defined primitives. Indeed, **natural languages** (such as English, German, and Latin) are distinguished from **formal languages** (such as programming languages) in that the latter are precisely defined by grammars, whereas the former evolved over time without formal grammatical analysis.

Translators

1) Total cost = price + shipping charge.

1) 156C
2) 166D
3) 5068
4) 306E
5) C000

The difference b/w (NL) and (FL). The programming languages are written in formal languages in which we are restricted by using some rules so we are not allowed to deviate from those rules that have been define by the (FL) however the (NL) are evolved when human communicate.

Copyright Virtual University of Pakistan

REDMI NOTE 8
AI QUAD CAMERA

Module 111

111. Programming Languages: Machine Independence

With the development of third-generation languages, the goal of machine independence was largely achieved. Since the statements in a third-generation language did not refer to the attributes of any particular machine, they could be compiled as easily for one machine as for another. A program written in a third-generation language could theoretically be used on any machine simply by applying the appropriate compiler.

Reality, however, has not proven to be this simple. When a compiler is designed, particular characteristics of the underlying machine are sometimes reflected as conditions on the language being translated. For example, the different ways in which machines handle I/O operations have historically caused the "same" language to have different characteristics, or dialects, on different machines. Consequently, it is often necessary to make at least minor modifications to a program to move it from one machine to another.

Compounding this problem of portability is the lack of agreement in some cases as to what constitutes the correct definition of a particular language. To aid in this regard, the American National Standards Institute and the International Organization for Standardization have adopted and published standards for many of the popular languages. In other cases, informal standards have evolved because of the popularity of a certain dialect of a language and the desire of other compiler writers to produce compatible products. However, even in the case of highly standardized languages, compiler designers often provide features, sometimes called language extensions, that are not part of the standard version of the language. If a programmer takes advantage of these features, the program produced will not be compatible with environments using a compiler from a different vendor. In the overall history of programming languages, the fact that third-generation languages fell short of true machine independence is actually of little significance for two reasons. First, they were close enough to being machine independent that software could be transported from one machine to another with relative ease. Second, the goal of machine independence turned out to be only a seed for more demanding goals. Indeed, the realization that machines could respond to such high-level statements as

assign TotalCost the value Price + ShippingCharge

led computer scientists to dream of programming environments that would allow humans to communicate with machines in terms of abstract concepts rather than forcing them to translate these concepts into machine-compatible form.

Moreover, computer scientists wanted machines that could perform much of the algorithm discovery process rather than just algorithm execution. The result has been an ever-expanding spectrum of programming languages that challenges a clear-cut classification in terms of generations.

language extensions → There are some extended parts of a compiler or extended instructions which are not part of original compilers.

REDMI NOTE 8
AI QUAD CAMERA

A method to do any task called paradigm.
A method to solve any problem also known that.

Module 112
Programming Languages: Imperative Paradigms

generation approach to classifying programming languages is based on a linear scale (Figure 91) on
h a language's position is determined by the degree to which the user of the language is freed from the
ds of computer gibberish and allowed to think in terms associated with the problem being solved. In
ct, the development of programming languages has not progressed in this manner but has developed
ng different paths as alternative approaches to the programming process (called **programming
radigms**) have surfaced and been pursued. Consequently, the historical development of programming
nguages is better represented by a multiple-track diagram as shown in Figure 92, in which different paths
ding from different paradigms are shown to emerge and progress independently. In particular, the figure
ects four paths representing the functional, object-oriented, imperative, and declarative paradigms, with
ious languages associated with each paradigm positioned in a manner that indicates their births relative
other languages. (It does not imply that one language necessarily evolved from a previous one.)

should note that although the paradigms identified in Figure 92 are called *programming* paradigms,
se alternatives have ramifications beyond the programming process. They represent fundamentally
fferent approaches to building solutions to problems and therefore affect the entire software development
ocess. In this sense, the term *programming paradigm* is a misnomer. A more realistic term would be
ftware development paradigm.

e imperative paradigm, also known as the **procedural paradigm,** represents the traditional approach
the programming process. It is the paradigm on which Python and our pseudocode are based as well as
e machine language. As the name suggests, the imperative paradigm defines the programming process to
the development of a sequence of commands that, when followed, manipulate data to produce the desired
ult. Thus, the imperative paradigm tells us to approach the programming process by finding an algorithm
olve the problem at hand and then expressing that algorithm as a sequence of commands.

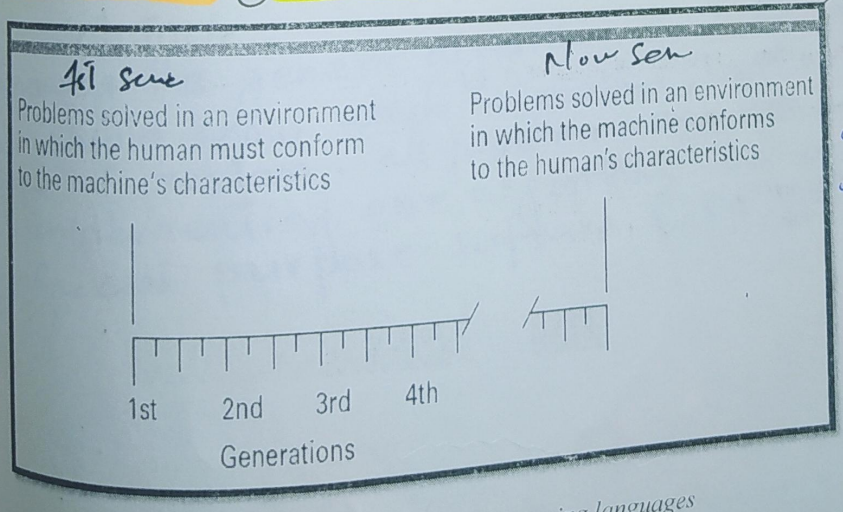


Figure 91: Generations of programming languages

we use a python, pseudocode and a machine language in imperative paradigm.

The process of finding the solution of a problem is called an algorithm.

Module 113

113. Programming Languages: Declarative Paradigms

In contrast to the imperative paradigm is the **declarative paradigm,** which asks a programmer to describe the problem to be solved rather than an algorithm to be followed. More precisely, a declarative programming system applies a pre-established general-purpose problem-solving algorithm to solve problems presented to it. In such an environment the task of a programmer becomes that of developing a precise statement of the problem rather than of describing an algorithm for solving the problem.

A major obstacle in developing programming systems based on the declarative paradigm is the need for an underlying problem-solving algorithm. For this reason, early declarative programming languages tended to be special-purpose in nature, designed for use in particular applications. For example, the declarative approach has been used for many years to simulate a system (political, economic, environmental, and so on) in order to test hypotheses or to obtain predictions. In these settings, the underlying algorithm is essentially the process of simulating the passage of time by repeatedly re-computing values of parameters (gross domestic product, trade deficit, and so on) based on the previously computed values. Thus, implementing a declarative language for such simulations requires that one first implement an algorithm that performs this repetitive function. Then the only task required of a programmer using the system is to describe the situation to be simulated. In this manner, a weather forecaster does not need to develop an algorithm for forecasting the weather but merely describes the current weather status, allowing the underlying simulation algorithm to produce weather predictions for the near future.

A tremendous boost was given to the declarative paradigm with the discovery that the subject of formal logic within mathematics provides a simple problem- solving algorithm suitable for use in a general-purpose declarative programming system. The result has been increased attention to the declarative paradigm and the emergence of **logic programming.**

obstacle:.
Knowing the generic algorithm and then implementing it
We should know which algorithm should be used for such type of problem. all problems can not be solved by implementing one algorithm So we need some special purpose softare like political, economic etc

ConceptsBuilder.com

REDMI NOTE 8
AI QUAD CAMERA

Module 114

114. Programming Languages: Functional Paradigm

Another programming paradigm is the **functional paradigm.** Under this paradigm a program is viewed as an entity that accepts inputs and produces outputs. Mathematicians refer to such entities as functions, which is the reason this approach is called the functional paradigm. Under this paradigm a program is constructed by connecting smaller predefined program units (predefined functions) so that each unit's outputs are used as another unit's inputs in such a way that the desired overall input-to-output relationship is obtained. In short, the programming process under the functional paradigm is that of building functions as nested complexes of simpler functions.

it's own characteristics

As an example, Figure 93 shows how a function for balancing your checkbook can be constructed from two simpler functions. One of these, called Find_sum, accepts values as its input and produces the sum of those values as its output. The other, called Find_diff, accepts two input values and computes their difference. The structure displayed in Figure 6.3 can be represented in the LISP programming language (a prominent functional programming language) by the expression

(Find_diff (Find_sum Old_balance Credits) (Find_sum Debits))

The nested structure of this expression (as indicated by parentheses) reflects the fact that the inputs to the function Find_diff are produced by two applications of Find_sum. The first application of Find_sum produces the result of adding all the Credits to the Old_balance. The second application of Find_sum computes the total of all Debits. Then, the function Find_diff uses these results to obtain the new checkbook balance.

To more fully understand the distinction between the functional and imperative paradigms, let us compare the functional program for balancing a checkbook to the following pseudocode program obtained by following the imperative paradigm:

Example

It is a imperative paradigm

```
Total_credits = sum of all Credits
Temp_balance = Old_balance + Total_credits
Total_debits = sum of all Debits
Balance = Temp_balance - Total_debits
```

REDMI NOTE 8
AI QUAD CAMERA

Collection of unit is called object. An object have a capability of performing action which is related to itself as well as from others object.

Module 115 Programming Languages: Object Oriented Paradigm

115. Programming Languages: Object Oriented Paradigm

Another programming paradigm (and the most prominent one in today's software development) is the **object-oriented paradigm**, which is associated with the programming process called **object-oriented programming (OOP)**. Following this paradigm, a software system is viewed as a collection of units, called **objects**, each of which is capable of performing the actions that are immediately related to itself as well as requesting actions of other objects. Together, these objects interact to solve the problem at hand.

What is objects?

example of object.

As an example of the object-oriented approach at work, consider the task of developing a graphical user interface. In an object-oriented environment, the icons that appear on the screen would be implemented as objects. Each of these objects would encompass a collection of functions (called **methods** in the object-oriented vernacular) describing how that object is to respond to the occurrence of various events, such as being selected by a click of the mouse button or being dragged across the screen by the mouse. Thus, the entire system would be constructed as a collection of objects, each of which knows how to respond to the events related to it.

have own

No use of algorithm

It based on algorithm.

To contrast the object-oriented paradigm with the imperative paradigm, consider a program involving a list of names. In the traditional imperative paradigm, this list would merely be a collection of data. Any program unit accessing the list would have to contain the algorithms for performing the required manipulations. In the object-oriented approach, however, the list would be constructed as an object that consisted of the list together with a collection of methods for manipulating the list. (This might include functions for inserting a new entry in the list, deleting an entry from the list, detecting if the list is empty, and sorting the list.) In turn, another program unit that needed to manipulate the list would not contain algorithms for performing the pertinent tasks. Instead, it would make use of the functions provided in the object. In a sense, rather than sorting the list as in the imperative paradigm, the program unit would ask the list to sort itself. Its significance in today's software development arena dictates that we include the concept of a class in this introduction. To this end, recall that an object can consist of data (such as a list of names) together with a collection of methods for per- forming activities (such as inserting new names in the list). These features must be described by statements in the written program. This description of the object's properties is called a **class**. Once a class has been constructed, it can be applied anytime an object with those characteristics is needed. Thus, several objects can be based on (that is, built from) the same class. Just like identical twins, these objects would be distinct entities but would have the same characteristics because they are constructed from the same template (the same class). (An object that is based on a particular class is said to be an **instance** of that class.)

Q Difference of List in imperative and OO-paradigm?
In imperative paradigm that list is contained as a list and if we want to sought or search within the list you need to write a particular algorithm for it however in OO-paradigm all functionality will be available along with the list in terms of object all of fuctionality will be available in the form of methods along with the list.

Q Class Such kind of description

class :- A class is method or variable in a particular kind of object.

REDMI NOTE 8
AI QUAD CAMERA

Primitive data types:.
The data types that are included as primitive in a Pro-Lan.
1) Integer 3) character
2) Float 4) Bolean

Module 116

Programming Languages: Variable and Data Types

116. When we store any data in a memory, then we give a location to it. and also descriptive name.

High-level programming languages allow locations in main memory to be referenced by descriptive names rather than by numeric addresses. Such a name is known as a **variable,** in recognition of the fact that by changing the value stored at the location, the value associated with the name changes as the program executes. Unlike Python, our example languages in this chapter require that variables be identified via a declarative statement prior to being used elsewhere in the program. These declarative statements also require that the programmer describe the type of data that will be stored at the memory location associated with the variable.

The data which we store in memory, such a type of data is called data type.

Such a type is known as a **data type** and encompasses both the manner in which the data item is encoded and the operations that can be performed on that data. For example, the type **integer** refers to numeric data consisting of whole numbers, probably stored using two's complement notation. Operations that can be performed on integer data include the traditional arithmetic operations and comparisons of relative size, such as determining whether one value is greater than another. The type **float** (sometimes called **real**) refers to numeric data that might contain values other than whole numbers, probably stored in floating-point notation. Operations performed on data of type float are similar to those performed on data of type integer. Recall, however, that the activity required for adding two items of type float differs from that for adding two items of type integer.

Suppose, then, that we wanted to use the variable WeightLimit in a program to refer to an area of main memory containing a numeric value encoded in two's complement notation. In the languages C, C++, Java, and C# we would declare our intention by inserting the statement

```
int WeightLimit;
```

toward the beginning of the program. This statement means "The name WeightLimit will be used later in the program to refer to a memory area containing a value stored in two's complement notation." Multiple variables of the same type can normally be declared in the same declaration statement. For example, the statement

```
int Height, Width;
```

would declare both Height and Width to be variables of type integer. Moreover, most languages allow a variable to be assigned an initial value when it is declared. Thus,

```
int WeightLimit = 100;
```

would not only declare WeightLimit to be a variable of type integer but also assign it the starting value 100. In contrast, dynamically typed languages like Python allow variables to be assigned without first declaring their type; such variables are checked for correct type later, when operations are performed upon them.

Other common data types include character and Boolean. The type character refers to data consisting of symbols, probably stored using ASCII or Unicode. Operations performed on such data include comparisons such as determining whether one symbol occurs before another in alphabetical order, testing to see whether one string of symbols appears inside another, and concatenating one string of symbols at the end of another to form one long string. The statement

```
char Letter, Digit;
```

Character

could be used in the languages C, C++, C#, and Java to declare the variables Letter and Digit to be of type character.

Copyright Virtual University of Pakistan

REDMI NOTE 8
AI QUAD CAMERA

The type **Boolean** refers to data items that can take on only the values true or false. Operations on data of type Boolean include inquiries as to whether the current value is true or false. For example, if the variable LimitExceeded was declared to be of type Boolean, then a statement of the form

if (LimitExceeded) then (...) else (...)

would be reasonable.

Primitive data type means character and integer.

The data types that are included as primitives in a programming language, such as int for integer and char for character, are called **primitive data types.** As we have learned, the data types such as: integer, float, character, and boolean are common primitives. Other data types that have not yet become widespread primitives include images, audio, video, and hypertext. However, types such as GIF, JPEG, and HTML might soon become as common as integer and float.

→ Integer variables are only those variables that can contain the integer values for example the numeric values the complete value like any value which is complete which does not contain any decimal point like any value 100, 20, 20,000 such are integers.
(int Price)

→ float variable can contain a data which can have decimal points in it. this means we can store information like 20.5 in this float variable. In memory location any value containing decimal point can be stored being reference by the Shipping price however it can not be stored in a memory location being reference by variable price.
(float shippingprice;)

→ character variable which will contain only one single character that is for example your grade wheater you get A, B, D these grades will be stored in a memory location that has been referenced by a variable grade (char grade;)

REDMI NOTE 8
AI QUAD CAMERA

Module 117

Programming Languages: Data Structure

In addition to data type, variables in a program are often associated with **data structure**, which is the conceptual shape or arrangement of data. For example, text is normally viewed as a long string of characters, whereas sales records might be envisioned as a rectangular table of numeric values, where each row represents the sales made by a particular employee and each column represents the sales made on a particular day.

A common data structure is the **array,** which is a block of elements of the same type such as a one-dimensional list, a two-dimensional table with rows and columns, or tables with higher dimensions. To establish such an array in a program, many programming languages require that the declaration statement naming the name of the array also specify the length of each dimension of the array. For example, Figure 94 displays the conceptual structure declared by the statement

```
int Scores[2][9];
```

in the language C, which means "The variable Scores will be used in the following program unit to refer to a two-dimensional array of integers having two rows and nine columns." The same statement in FORTRAN would be written as

```
INTEGER Scores (2, 9)
```

Once an array has been declared, it can be referenced elsewhere in the program by its name, or an individual element can be identified by means of integer values called **indices** that specify the row, column, and so on, as desired. However, the range of these indices varies from language to language. For example, in C (and its derivatives C++, Java, and C#) indices start at 0, meaning that the entry in the second row and fourth column of the array called Scores (as declared above) would be referenced by Scores[1][3], and the entry in the first row and first column would be Scores[0][0]. In contrast, indices start at 1 in a FORTRAN program so the entry in the second row and fourth column would be referenced by Scores(2,4) (see again Figure 94).

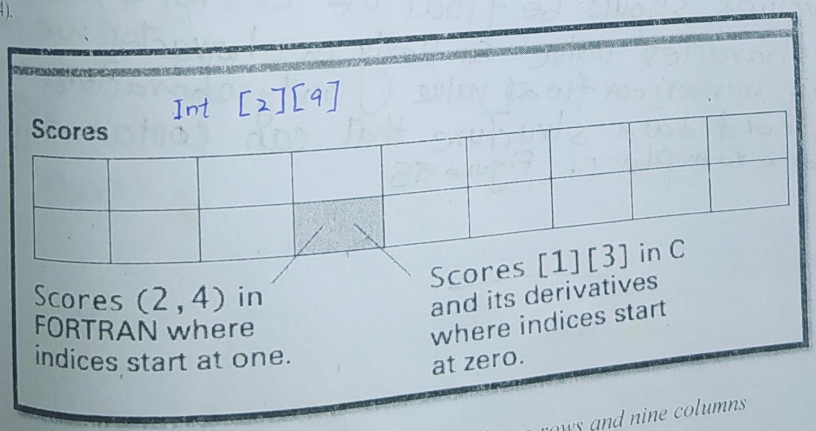


Figure 94: A two-dimensional array with two rows and nine columns

In contrast to an array in which all data items are the same type, an **aggregate type** (also called a **structure,** a **record,** or sometimes a **heterogeneous array**) is a block of data in which different elements can have different types. For instance, a block of data referring to an employee might consist of an entry called Name

Copyright Virtual University of Pakistan

Module 118

118. Programming Languages: Assignment Statement

Once the special terminology to be used in a program (such as the variables and constants) has been declared, a programmer can begin to describe the algorithms involved. This is done by means of imperative statements. The most basic imperative statement is the **assignment statement**, which requests that a value be assigned to a variable (or more precisely, stored in the memory area identified by the variable). Such a statement normally takes the syntactic form of a variable, followed by a symbol representing the assignment operation, and then by an expression indicating the value to be assigned. The semantics of such a statement is that the expression is to be evaluated and the result stored as the value of the variable. For example, the statement:

In Computer, = this value not known is equal But here it is known as assign to.

```
Z=X+Y;
```

in C, C++, C#, and Java requests that the sum of X and Y be assigned to the variable Z. The semicolon at the end of the line, which is used to separate statements in many imperative languages, is the only syntactic difference from an equivalent Python assignment statement. In some other languages (such as Ada) the equiva- lent statement would appear as:

(C++)

```
Z:=X+Y;
```

example marks = 10 Ten value is being stored at the memory "marks is assigned by 10." being referenced by variable name location which is "marks".

```
int price;
Price = 10;
cout << old value of price is" << price
Price = 1000;
cout << "new value of price is" << price;
```

in javascript.

101000.

REDMI NOTE 8
AI QUAD CAMERA

Module 119

119. Programming Languages: Control Structures (if-statement)

changes
A **control statement** alters the execution sequence of the program. Of all the programming constructs,

if (condition) if we want to execute some statement or not. that this statement should be executed if a particular condition is fulfilled

StatementA

else

StatementB.

For example, if a student's gets more than or equal to 50 marks, the student passes the examination. To denote this using the if-statement, we will proceed as follow:

```
if (marks>=50)
You have passed the examination
Else / otherwise
You have failed the examination
```

```
cout << "Hello to VU " >> {if (marks> =50)
cout << " you are passed " >
```

REDMI NOTE 8
AI QUAD CAMERA

Module 120

120. Programming Languages: Control Structures (if-statement examples)

In this module, we will learn another example. Suppose a university wants to give a scholarship, if a student gets more than 3.0 CGPA in a given semester. We can write the program in the following way and you have also seen its implementation in the videos in the online compiler.

```
float f=3.5;

if (CGPA>=3.0)
cout<<"Give Scholarship";

else

cout<<"Sorry you do not qualify for the scholarship";
```

REDMI NOTE 8
AI QUAD CAMERA

Using loop you can control the flow of your instructions. So it will not be executed one instruction then 2nd then instead loop will check the condition. If the condition is true then it will execute the same instructions again and again and if you have inserted

Module 121

121. Programming Languages: Control Structures (Loops)

Repeats set of instruction.

There is another type of control structure known as loop. The loop control structure iterates a set of instructions based on the provided condition.

Consider the following example, we are interested to print the counting from 1 to 5, One way of doing is as follows:

```
cout<<"1";
cout<<"2";
cout<<"3";
cout<<"4";
cout<<"5";
```

Doing this using loop is easier, you just need to give one statement that is printing counting and you need to tell that how many times, you want to do it. One can write the loop as follows:

```
int i=1;
while(i<=5)
{
cout<<i;
i=i+1;
}
```

This loop will execute 5 times as follows:

First the value of "i" would be 1. The condition would be checked and based on the condition as true (i.e 1<=5), the loop will continue and will print the value of "i" which is 1. Then it will increment the value of "i". The new value of "i" would be 2. Then again condition would be checked, and the condition would be true as 2<=5. This will print "2" on the screen and will continue for "i" as 3, 4, and 5, and will print 3, 4, 5 on the screen as well. When the value of "i" would be 6, the loop will find the condition as false (6<=5). This would be the termination for the loop.

For table of 5.

```
int i=1;
while (i<<10)
{
  cout<<"5 *"<<i<<" = "<<5*i<<"\n";
```

REDMI NOTE 8
AI QUAD CAMERA

Sometimes we do not want to execute one code at once instead we want to execute many different codes simultaneously and such a parallel processing or concurrent processing

Module 122

When a computer perform a multiple activities, same time.

122. Programming Languages: Programming Concurrent Activities

It is a technique in which two or more processes that in a This is concurrent activities.

Suppose we were asked to design a program to produce animation for an action computer game involving multiple attacking enemy spaceships. One approach would be to design a single program that would control the entire animation screen. Such a program would be charged with drawing each of the spaceships, which (if the animation is to appear realistic) would mean that the program would have to keep up with the individual characteristics of numerous spacecraft. An alternate approach would be to design a program to control the animation of a single spaceship whose characteristics are determined by parameters assigned at the beginning of the program's execution. Then the animation could be constructed by creating multiple activations of this program, each with its own set of parameters. By executing these activations simultaneously, we could obtain the illusion of many individual spaceships streaking across the screen at the same time.

Such simultaneous execution of multiple activations is called **parallel processing** or **concurrent processing.** True parallel processing requires multiple CPU cores, one to execute each activation. When only one CPU is available, the illusion of parallel processing is obtained by allowing the activations to share the time of the single processor in a manner similar to that implemented by multiprogramming systems.

Many modern computer applications are more easily solved in the context of parallel processing than in the more traditional context involving a single sequence of instructions. In turn, newer programming languages provide syntax for expressing the semantic structures involved in parallel computations. The design of such a language requires the identification of these semantic structures and the development of a syntax for representing them.

Each programming language tends to approach the parallel processing paradigm from its own point of view, resulting in different terminology. For example, what we have informally referred to as an *activation* is called a *task* in the Ada vernacular and a *thread* in Java. That is, in an Ada program, simultaneous actions are performed by creating multiple *tasks*, whereas in Java one creates multiple *threads*. In either case, the result is that multiple activities are generated and executed in much the same way as processes under the control of a multitasking operating system. We will adopt the Java terminology and refer to such "processes" as threads.

Perhaps the most basic action that must be expressed in a program involving parallel processing is that of creating new threads. If we want multiple activations of the spaceship program to be executed at the same time, we need a syntax for saying so. Such spawning of new threads is similar to that of requesting the execution of a traditional function. The difference is that, in the traditional setting, the program unit that requests the activation of a function does not progress any further until the requested function terminates (recall Figure 6.8), whereas in the parallel context the requesting program unit continues execution while the requested function performs its task (Figure 96). Thus, to create multiple space- ships streaking across the screen, we would write a main program that simply generates multiple activations of the spaceship program, each provided with the parameters describing the distinguishing characteristics of that spaceship.

in Activation:: Different methodologies and naming conventions different programming languages

✓ Task in Ada

✓ Thread in java

REDMI NOTE 8
AI QUAD CAMERA

```
int a=5
int b=2
Cout<<a-b;
```


Example: 2 | 3 (1) — 2 — 1

Module 123

123. Programming Languages: Arithmetic Operators Examples

In C++, the remaining 1 is not known a remainder but it is known as a modulus %

C-language has the following arithmetic operators:

+	Addition
-	Subtraction
*	Multiplication
/	Division
%	Modulus

+, -, and * are the same as used in the mathematics. However, the "/" has a difference. If one of the operands is decimal number, then it results in the same way as in mathematics, for example:

/ ← 2 | 5 (2) — 4 — 1

5.0/2.0 would result into 2.5.

However, when both operands are integers, then it would truncate the decimal point and

5/2 would result into 2.

% ↙

The remaining "1" can be acquired by using the modulus operator (%).

5%2 would give 1.

If a <= b

Module 124

124. Programming Languages: Relational Operators Examples

C-language has the following relational operators:

<	Less than
<=	Less than or equal to
>	Greater than
>=	Greater than or equal to
==	Equal to
!=	Not Equal to

C++ Relational Operators are used to compare values of two variables. Here in example we used the operators in if statement.

Now if the result after comparison of two variables is True, then if statement returns value 1.

And if the result after comparison of two variables is False, then if statement returns value 0.

```
#include<iostream>
using namespace std;
void main()
{
  int a=10,b=20,c=10;

  if(a>b)
```


Copyright Virtual University of Pakistan

REDMI NOTE 8
AI QUAD CAMERA

Module 125

125. Programming Languages: Logical Operators Examples

1. C-language Logical Operators are used if we want to compare more than one condition.
2. Depending upon the requirement, proper logical operator is used.
3. Following table shows us the different C++ operators available

IMP

Operators	Name of the Operator	Type
&&	AND Operator	Binary
||	OR Operator	Binary
!	NOT Operator	Unary

According to names of the Logical Operators, the condition satisfied in following situation and expected outputs are given

Operator	Output
AND	Output is 1 only when conditions on both sides of Operator become True
OR	Output is 0 only when conditions on both sides of Operator become False
NOT	It gives inverted Output

T = 1
F = 0

Let us look at all logical operators with example-

if(CGPA<3.88 Age<20)

```
#include<iostream>
using namespace std;

int main()
{
  int num1=30;
  int num2=40;

  if(num1>=40 || num2>=40)
    cout<<"OR If Block Gets Executed"<<endl;

  if(num1>=20 && num2>=20)
    cout<<"AND If Block Gets Executed"<<endl;

  if(!(num1>=40))
    cout<<"NOT If Block Gets Executed"<<endl;

  return 0;
}
```

Output:

```
OR If Block Gets Executed
```


Copyright Virtual University of Pakistan

we want to build software which can be used by the humans with proper satisfaction level

Module 126

126. Software Engineering: Software Engineering Discipline

To appreciate the problems involved in software engineering, it is helpful to select a large complex device (an automobile, a multistory office building, or perhaps a cathedral) and imagine being asked to design it and then to supervise its construction. How can you estimate the cost in time, money, and other resources to complete the project? How can you divide the project into manageable pieces? How can you ensure that the pieces produced are compatible? How can those working on the various pieces communicate? How can you measure progress? How can you cope with the wide range of detail (the selection of the doorknobs, the design of the gargoyles, the availability of blue glass for the stained glass windows, the strength of the pillars, the design of the duct work for the heating system)? Questions of the same scope must be answered during the development of a large software system.

Because engineering is a well-established field, you might think that there is a wealth of previously developed engineering techniques that can be useful in answering such questions. This reasoning is partially true, but it overlooks fundamental differences between the properties of software and those of other fields of engineering. These distinctions have challenged software engineering projects, leading to cost overruns, late delivery of products, and dissatisfied customers. In turn, identifying these distinctions has proven to be the first step in advancing the software engineering discipline.

One such distinction involves the ability to construct systems from generic prefabricated components. Traditional fields of engineering have long benefited from the ability to use "off-the-shelf" components as building blocks when constructing complex devices. The designer of a new automobile does not have to design a new engine or transmission but instead uses previously designed versions of these components. Software engineering, however, lags in this regard. In the past, previously designed software components were domain specific that is, their internal design was based on a specific application and thus their use as generic components was limited. The result is that complex software systems have historically been built from scratch. As we will see in this chapter, significant progress is being made in this regard, although more work remains to be done.

Another distinction between software engineering and other engineering disciplines is the lack of quantitative techniques, called **metrics,** for measuring the properties of software. For example, to project the cost of developing a software system, one would like to estimate the complexity of the proposed product, but methods for measuring the "complexity" of software are evasive. Similarly, evaluating the quality of a software product is challenging. In the case of mechanical devices, an important measure of quality is the mean time between failures, which is essentially a measurement of how well a device endures wear. Software, in contrast, does not wear out, so this method of measuring quality is not as applicable in software engineering.

The difficulties involved in measuring software properties in a quantitative manner is one of the reasons that software engineering has struggled to find a rigorous footing in the same sense as mechanical and electrical engineering. Whereas these latter subjects are founded on the established science of physics, software engineering continues to search for its roots.

Thus, research in software engineering is currently progressing on two levels. Some researchers, sometimes called practitioners, work toward developing techniques for immediate application, whereas others, called theoreticians, search for underlying principles and theories on which more stable techniques can someday be constructed. Being based on a subjective foundation, many methodologies developed and promoted by practitioners in the past have been replaced by other approaches that may themselves become obsolete with time. Meanwhile, progress by theoreticians continues to be slow.

REDMI NOTE 8
AI QUAD CAMERA

Copyright Virtual University of Pakistan

The need for progress by both practitioners and theoreticians is enormous. Our society has become addicted to computer systems and their associated software. Our economy, healthcare, government, law enforcement, transportation, and defense depend on large software systems. Yet there continue to be major problems with the reliability of these systems. Software errors have caused such disasters and near disasters as the rising moon being interpreted as a nuclear attack, a one-day loss of $5 million by the Bank of New York, the loss of space probes, radiation overdoses that have killed and paralyzed, and the simultaneous disruption of telephone communications over large regions.

This is not to say that the situation is all bleak. Much progress is being made in overcoming such problems as the lack of prefabricated components and metrics. Moreover, the application of computer technology to the software development process, resulting in what is called **computer-aided software engineering (CASE)**, is continuing to streamline and otherwise simplify the software development process. CASE has led to the development of a variety of computerized systems, known as **CASE tools**, which include project planning systems (to assist in cost estimation, project scheduling, and personnel allocation), project management systems (to assist in monitoring the progress of the development project), documentation tools (to assist in writing and organizing documentation), prototyping and simulation systems (to assist in the development of prototypes), interface design systems (to assist in the development of GUIs), and programming systems (to assist in writing and debugging programs). Some of these tools are little more than the word processors, spreadsheet software, and email communication systems that were originally developed for generic use and adopted by software engineers. Others are quite sophisticated packages designed primarily for the software engineering environment. Indeed, systems known as **integrated development environments (IDEs)** combine tools for developing software (editors, compilers, debugging tools, and so on) into a single, integrated package. Prime examples of such systems are those for developing applications for smartphones. These not only provide the programming tools necessary to write and debug the software but also provide simulators that, by means of graphical displays, allow a programmer to see how the software being developed would actually perform on a phone.

Wrong estimations
1) could lead to cost overruns
2) late delivery of products
3) Dissatisfied customers

Diference b/w Software Engineering and Engineering.
- Using off the shelf components (making new vehicle does not require to develop engine)
- Quantitative techniques called metrices eg measuring complexity of software.

REDMI NOTE 8
AI QUAD CAMERA

Module 127

127. Software Engineering: Software Life cycle

The most fundamental concept in software engineering is the software life cycle. The software life cycle is shown in Figure 97. This figure represents the fact that once software is developed, it enters a cycle of being used and maintained—a cycle that continues for the rest of the software's life. Such a pattern is common for many manufactured products as well. The difference is that, in the case of other products, the maintenance phase tends to be a repair process, whereas in the case of software, the maintenance phase tends to consist of correcting or updating. Indeed, software moves into the maintenance phase because errors are discovered, changes in the software's application occur that require corresponding changes in the software, or changes made during a previous modification are found to induce problems elsewhere in the software.

Process of maintenance

Regardless of why software enters the maintenance phase, the process requires that a person (often not the original author) study the underlying program and its documentation until the program, or at least the pertinent part of the program, is understood. Otherwise, any modification could introduce more problems than it solves. Acquiring this understanding can be a difficult task, even when the software is well-designed and documented. In fact, it is often within this phase that a piece of software is discarded under the pretense (too often true) that it is easier to develop a new system from scratch than to modify the existing package successfully.

Experience has shown that a little effort during the development of software can make a tremendous difference when modifications are required.

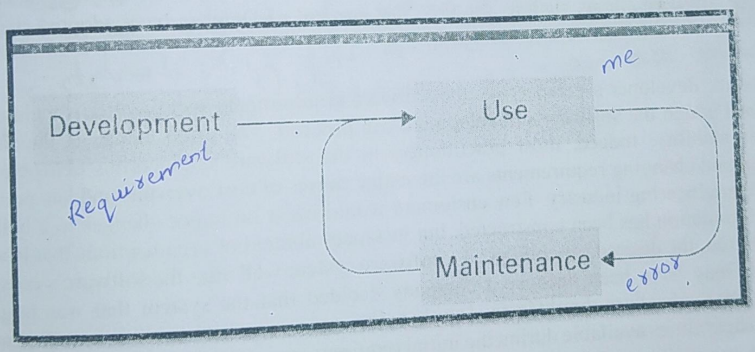


Figure 97: The software life cycle

When Software is moved to maintenance?
- Errors are discovered
- Changes in Software application occurs
- Changes done in previous modification introduce the errors

Copyright Virtual University of Pakistan

Module 128

128. Software Engineering: Requirement Analysis Phase

The major steps in the traditional software development life cycle are requirements analysis, design, implementation, and testing (Figure 98).

Requirements Analysis: The software life cycle begins with requirements analysis—the goal of which is to specify what services the proposed system will provide, to identify any conditions (time constraints, security, and so on) on those services, and to define how the outside world will interact with the system.

Requirements analysis involves significant input from the **stakeholders** (future users as well as those with other ties, such as legal or financial interests) of the proposed system. In fact, in cases where the ultimate user is an entity, such as a company or government agency, that intends to hire a software developer for the actual execution of the software project, requirements analysis may start by a feasibility study conducted solely by the user. In other cases, the software developer may be in the business of producing **commercial off-the-shelf (COTS)** software for the mass market, perhaps to be sold in retail stores or downloaded via the Internet. In this setting the user is a less precisely defined entity, and requirements analysis may begin with a market study by the software developer. In any case, the requirements analysis process consists of compiling and analyzing the needs of the software user; negotiating with the project's stakeholders over trade-offs between wants, needs, costs, and feasibility; and finally developing a set of requirements that identify the features and services that the finished software system must have. These requirements are recorded in a document called a **software requirements specification**. In a sense, this document is a written agreement between all parties concerned, which is intended to guide the software's development and provide a means of resolving disputes that may arise later in the development process. The significance of the software requirements specification is demonstrated by the fact that professional organizations such as IEEE and large software clients such as the U.S. Department of Defense have adopted standards for its composition.

From the software developer's perspective, the software requirements specification should define a firm objective toward which the software's development can proceed. Too often, however, the document fails to provide this stability. Indeed, most practitioners in the software engineering field argue that poor communication and changing requirements are the major causes of cost overruns and late product delivery in the software engineering industry. Few customers would insist on major changes to a building's floor plan once the foundation has been constructed, but instances abound of organizations that have expanded, or otherwise altered, the desired capabilities of a software system well after the software's construction was underway. This may have been because a company decided that the system that was originally being developed for only a subsidiary should instead apply to the entire corporation or that advances in technology supplanted the capabilities available during the initial requirements analysis. In any case, software engineers have found that straightforward and frequent communication with the project's stakeholders is mandatory.

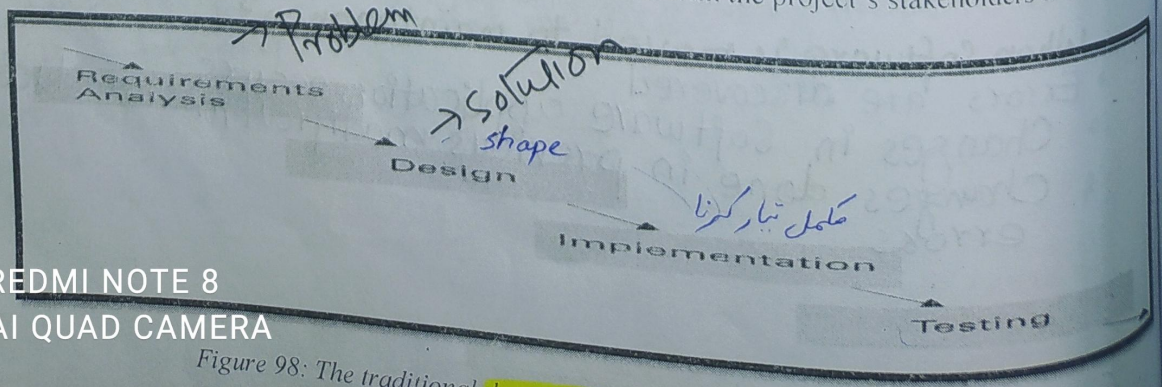


Figure 98: The traditional develop

REDMI NOTE 8
AI QUAD CAMERA

Module 129

29. Software Engineering: Design Phase

ضرورت RA/ Design.

Whereas requirements analysis provides a description of the proposed software product, design involves creating a plan for the construction of the proposed system. In a sense, requirements analysis is about identifying the problem to be solved, while design is about developing a solution to the problem. From a day- person's perspective, requirements analysis is often equated with deciding *what* a software system is to do, whereas design is equated with deciding *how* the system will do it. Although this description is enlightening, many software engineers argue that it is flawed because, in actuality, there is a lot of *how* considered during requirements analysis and a lot of *what* considered during design.

Desing phase

It is in the design stage that the internal structure of the software system is established. The result of the design phase is a detailed description of the software system's structure that can be converted into programs.

office building example

If the project were to construct an office building rather than a software system, the design stage would consist of developing detailed structural plans for a building that meets the specified requirements. For example, such plans would include a collection of blueprints describing the proposed building at various levels of detail. It is from these documents that the actual building would be constructed. Techniques for developing these plans have evolved over many years and include standardized notational systems and numerous modeling and diagramming methodologies.

(In Software design:)

In COTS:. If instance you are not making Something for a Specific user. instead you are making a service or developing for a mass market that is going to be sold in retail stores or can be downloded via internet.

Copyright Virtual University of Pakistan

Module 130

130. Software Engineering: Implementation Phase

Implementation involves the actual writing of programs, creation of data files, and development of databases. It is at the implementation stage that we see the distinction between the tasks of a **software analyst** (sometimes referred to as a system analyst) and a **programmer**. The former is a person involved with the entire development process, perhaps with an emphasis on the requirements analysis and design steps. The latter is a person involved primarily with the implementation step. In its narrowest interpretation, a programmer is charged with writing programs that implement the design produced by a software analyst. Having made this distinction, we should note again that there is no central authority controlling the use of terminology throughout the computing community. Many who carry the title of software analyst are essentially programmers, and many with the title programmer (or perhaps senior programmer) are actually software analysts in the full sense of the term. This blurring of terminology is founded in the fact that today the steps in the software development process are often intermingled.

Software Analyst vs programmer.

Software Analyst is someone who is mainly focusing the requirement analysis phase and the design phase however the programmer is mainly focusing on the development phase but in computer science or in is CS community there is no central authority who is controlling all of such terminology

Whatever has been build using the Implementation phase and whatever has been designed using design and whatever requirements has been gathered needs to be check and all of such things need to be identified all of the bugs.

Module 131

131. Software Engineering: Testing Phase

In the traditional development phase of the past, testing was essentially equated with the process of debugging programs and confirming that the final software product was compatible with the software requirements specification. Today, however, this vision of testing is considered far too narrow. Programs are not the only artifacts that are tested during the software development process. Indeed, the result of each intermediate step in the entire development process should be "tested" for accuracy. Testing is now recognized as only one segment in the overall struggle for quality assurance, which is an objective that permeates the entire software life cycle. Thus, many software engineers argue that testing should no longer be viewed as a separate step in software development, but instead it, and its many manifestations, should be incorporated into the other steps, producing a three-step development process whose components might have names such as "requirements analysis *and confirmation*," "design *and validation*," and "implementation *and testing*."

Modern testing

Need for better testing methodoligies.

Unfortunately, even with modern quality assurance techniques, large software systems continue to contain errors, even after significant testing. Many of these errors may go undetected for the life of the system, but others may cause major malfunctions. The elimination of such errors is one of the goals of software engineering. The fact that they are still prevalent indicates that a lot of research remains to be done.

Modern Testing:.

Testing should not be considered as Separate step:

- ✓ Requirements analysis and confirmation
- ✓ Design and validation
- ✓ Implementation and testing.

Copyright Virtual University of Pakistan

REDMI NOTE 8
AI QUAD CAMERA

Module 132

132. Software Engineering: Software Engineering Methodologies (I)

Early approaches to software engineering insisted on performing requirements analysis, design, implementation, and testing in a strictly sequential manner. The belief was that too much was at risk during the development of a large software system to allow for variations. As a result, software engineers insisted that the entire requirements specification of the system be completed before beginning the design and, likewise, that the design be completed before beginning implementation. The result was a development process now referred to as the **water- fall model,** an analogy to the fact that the development process was allowed to flow in only one direction.

In recent years, software engineering techniques have changed to reflect the contradiction between the highly structured environment dictated by the water- fall model and the "free-wheeling," trial-and-error process that is often vital to creative problem solving. This is illustrated by the emergence of the **incremental model** for software development. Following this model, the desired software system is constructed in increments, the first being a simplified version of the final product with limited functionality. Once this version has been tested and perhaps evaluated by the future user, more features are added and tested in an incremental manner until the system is complete. For example, if the system being developed is a patient records system for a hospital, the first increment may incorporate only the ability to view patient records from a small sample of the entire record system. Once that version is operational, additional features, such as the ability to add and update records, would be added in a stepwise manner.

Another model that represents the shift away from strict adherence to the waterfall model is the **iterative model,** which is similar to, and in fact sometimes equated with, the incremental model, although the two are distinct. Whereas the incremental model carries the notion of *extending* each preliminary version of a product into a larger version, the iterative model encompasses the concept of *refining* each version. In reality, the incremental model involves an underlying iterative process, and the iterative model may incrementally add features.

A significant example of iterative techniques is the **rational unified process (RUP,** rhymes with "cup") that was created by the Rational Software Corporation, which is now a division of IBM. RUP is essentially a software development paradigm that redefines the steps in the development phase of the software life cycle and provides guidelines for performing those steps. These guidelines, along with CASE tools to support them, are marketed by IBM. Today, RUP is widely applied throughout the software industry. In fact, its popularity has led to the development of a nonproprietary version, called the **unified process** that is available on a noncommercial basis.

Water fall Method:
→ Entire preceding phase need to be completely done before the start of next phase. Its disadvantage is if there is a requirement arising at later stages we cannot go to the previous page.

REDMI NOTE 8
AI QUAD CAMERA

Prototype → incomplete version of the proposed system built and evaluated in iterative and incremental version.

When we made any model and give it to a user then a user give feedback. It is called prototyping.

Module 133
Software Engineering: S ftware Engineering Methodologies (II)

Incremental and iterative models sometimes make use of the trend in software development toward **prototyping** in which incomplete versions of the proposed system, called prototypes, are built and evaluated. In the case of the increme tal model these prototypes evolve into the complete, final system, a process known as **evolutionary prototyping.** In a more iterative situation, the prototypes may be discarded in favor of a fresh implementation of he final design. This approach is known as **throwaway prototyping.** An example that normally falls with n this throwaway category is **rapid prototyping** in which a simple example of the proposed system is qu kly constructed in the early stages of development. Such a prototype may consist of only a few screen im ges that give an indication of how the system will interact with its users and what capabilities it will hav The goal is not to produce a working version of the product but to obtain a demonstration tool that can b used to clarify communication between the parties involved in the software development process. For e ample, rapid prototypes have proved advantageous in clarifying system requirements during requireme s analysis or as aids during sales presentations to potential clients.

A less formal incarnation of incremental and iterative ideas that has been used for years by computer enthusiasts/hobbyists is known as **open- ource development.** This is the means by which much of today's free software is produced. Perhaps the m st prominent example is the Linux operating system whose open-source development was originally led by Linus Torvalds. The open-source development of a software package proceeds as follows: A single a hor writes an initial version of the software (usually to fulfill his or her own needs) and posts the source ode and its documentation on the Internet. From there it can be downloaded and used by others witho charge. Because these other users have the source code and documentation, they are able to modify o enhance the software to fit their own needs or to correct errors that they find. They report these change to the original author, who incorporates them into the posted version of the software, making this exten ed version available for further modifications. In practice, it is possible for a software package to evolve t rough several extensions in a single week.

Perhaps the most pronounced shift fro the waterfall model is represented by the collection of methodologies known as **agile methods,** e ch of which pro- poses early and quick implementation on an incremental basis, responsiveness to ch nging requirements, and a reduced emphasis on rigorous requirements analysis and design. One e ample of an agile method is **extreme programming (XP).** Following the XP model, software is dev loped by a team of less than a dozen individuals working in a communal workspace where they freely sh re ideas and assist each other in the development project. The software is developed incrementally by me ns of repeated daily cycles of informal requirements analysis, designing, implementing, and testing. Thu , new expanded versions of the software package appear on a regular basis, each of which can be evaluated by the project's stakeholders and used to point toward further increments. In summary, agile methods ar characterized by flexibility, which is in stark contrast to the waterfall model that conjures the image of managers and programmers working in individual offices while rigidly performing well-defined portions of the overall software development task.

The contrasts depicted by comparing the waterfall model and XP reveal the breadth of methodologies that are being applied to the software development process in the hopes of finding better ways to construct reliable software in an efficient manner. Research in the field is an ongoing process. Progress is being made, but much work remains to be done.

Open-source development so when someones to contribute to that software so they write their code and inform the author which updates the next version and which is available again to the community.

Copyright Virtual University of Pakistan

REDMI NOTE 8
AI QUAD CAMERA

Module 134

134. Software Engineering: Software Engineering Methodologies (II)

A key point is that to modify software one must understand the program or at least the pertinent parts of the program. Gaining such an understanding is often difficult enough in the case of small programs and would be close to impossible when dealing with large software systems if it were not for **modularity** that is, the division of software into manageable units, generically called **modules**, each of which deals with only a part of the software's overall responsibility.

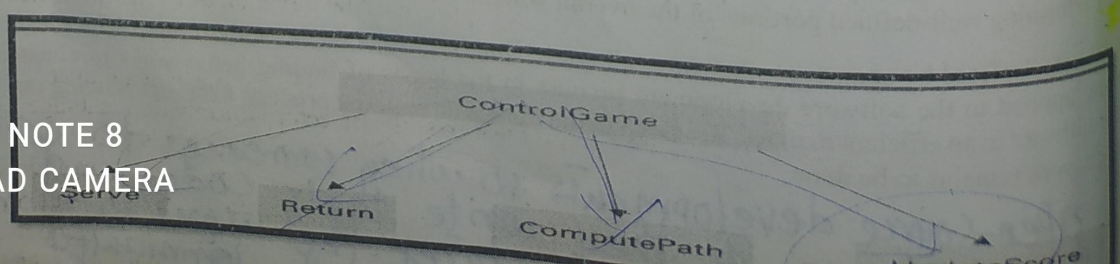
Modules come in a variety of forms. We have already seen that in the context of the imperative paradigm, modules appear as functions. In contrast, the object-oriented paradigm uses objects as the basic modular constituents. These distinctions are important because they determine the underlying goal during the initial software design process. Is the goal to represent the overall task as individual, manageable processes or to identify the objects in the system and understand how they interact?

To illustrate, let us consider how the process of developing a simple modular program to simulate a tennis game might progress in the imperative and the object-oriented paradigms. In the imperative paradigm we begin by considering the actions that must take place. Because each volley begins with a player serving the ball, we might start by considering a function named Serve that (based on the player's characteristics and perhaps a bit of probability) would compute the initial speed and direction of the ball. Next, we would need to determine the path of the ball (Will it hit the net? Where will it bounce?). We might plan on placing these computations in another function named ComputePath. The next step might be to determine if the other player is able to return the ball, and if so, we must compute the ball's new speed and direction. We might plan on placing these computations in a function named Return.

Continuing in this fashion, we might arrive at the modular structure depicted by the **structure chart** shown in Figure 99, in which functions are represented by rectangles and function dependencies (implemented by function calls) are represented by arrows. In particular, the chart indicates that the entire game is overseen by a function named ControlGame, and to perform its task, ControlGame calls on the services of the functions Serve, Return, ComputePath, and UpdateScore.

Note that the structure chart does not indicate how each function is to perform its task. Rather, it merely identifies the functions and indicates the dependencies among the functions. In reality, the function ControlGame might perform its task by first calling the Serve function, then repeatedly calling on the functions ComputePath and Return until one reports a miss, and finally calling on the services of UpdateScore before repeating the whole process by again calling on Serve.

At this stage we have obtained only a very simplistic outline of the desired program, but our point has already been made. In accordance with the imperative paradigm, we have been designing the program by considering the activities that must be performed and are therefore obtaining a design in which the modules are functions.



REDMI NOTE 8
AI QUAD CAMERA

Each module should be self-contained and it should not be dependent on other module.

concepts builder

Module 135

135. Software Engineering: Coupling

Advantage of modularity.

We have introduced modularity as a way of producing manageable software. The idea is that any future modification will likely apply to only a few of the modules, allowing the person making the modification to concentrate on that portion of the system rather than struggling with the entire package. This, of course, depends on the assumption that changes in one module will not unknowingly affect other modules in the system. Consequently, a goal when designing a modular system should be to maximize independence among modules or, in other words, to minimize the linkage between modules (known as intermodule **coupling**). Indeed, one metric that has been used to measure the complexity of a software system (and thus obtain a means of estimating the expense of maintaining the software) is to measure its intermodule coupling.

Intermodule coupling occurs in several forms. One is **control coupling**, which occurs when a module passes control of execution to another, as in a function call. The structure chart in Figure 99 represents the control coupling that exists between functions. In particular, the arrow from the module ControlGame to Serve indicates that the former passes control to the latter. It is also control coupling that is represented in Figure 100, where the arrows trace the path of control as it is passed from object to object.

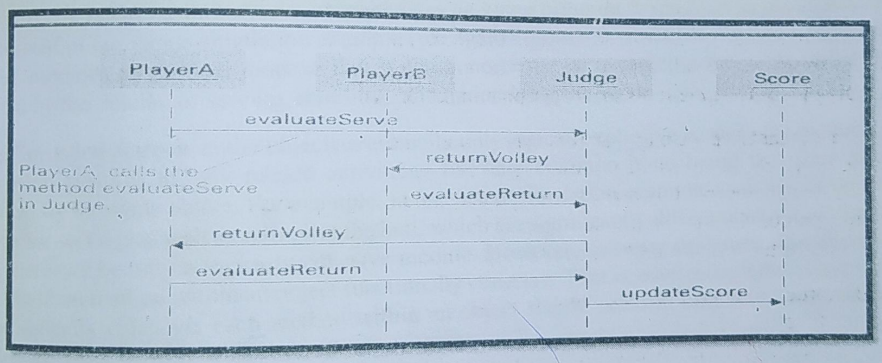


Figure 100: The interaction between objects resulting from PlayerA's serve.

Another form of intermodule coupling is **data coupling**, which refers to the sharing of data between modules. If two modules interact with the same item of data, then modifications made to one module may affect the other, and modifications to the format of the data itself could have repercussions in both modules.

Data coupling between functions can occur in two forms. One is by explicitly passing data from one function to another in the form of parameters. Such coupling is represented in a structure chart by an arrow between the functions that is labeled to indicate the data being passed. The direction of the arrow indicates the direction in which the item is transferred.

Similar data coupling occurs between objects in an object-oriented design. For example, when PlayerA asks the object Judge to evaluate its serve (see Figure 100), it must pass the trajectory information to Judge. On the other hand, one of the benefits of the object-oriented paradigm is that it inherently tends to reduce data coupling between objects to a minimum. This is because the methods within an object tend to include all those functions that manipulate the object's internal data. For example, the object PlayerA will contain information regarding that player's characteristics as well as all the methods that require that information. In turn, there is no need to pass that information to other objects and thus interobject data coupling is minimized.

Copyright Virtual University of Pakistan

REDMI NOTE 8
AI QUAD CAMERA